

StudySync Distributed Systems — Failure Analysis & Remediation

PDC Assignment | Muhammad Sameer | BSCS23176

Part 1 — Failure Analysis

Problem 1 · Lost Update (Synchronization)

The root cause is the absence of any concurrency control at the database layer. When two users load the same document, each client receives a snapshot of the current state. Both then issue a **PUT /document/{id}** with their independently modified payloads. FastAPI processes these as two separate, unaware transactions. Because neither read carried a version token, the second write simply overwrites the first — a textbook *Lost Update anomaly*. The problem surfaces at the gap between the read and the write: the window during which another writer can silently invalidate the first writer's assumptions.

Problem 2 · Dropped Webhook (Coordination)

Clerk delivers subscription-cancellation events as HTTP POST webhooks to the backend. The current handler is fire-and-forget: it receives the event, updates the database, and replies 200 OK. This design has no durability guarantee. If the network drops the POST before it reaches the server, or if the server crashes mid-processing, the event is silently lost. Because Clerk's default retry policy is limited, and because there is no idempotency guard or event log, the user's subscription record is never updated — a permanent state inconsistency between Clerk's authoritative record and the application database.

Problem 3 · Synchronous LLM Call (Fault Tolerance)

The */ask* endpoint issues an *httpx* call to an external LLM API and awaits the response inline. When that API becomes slow or unresponsive, the coroutine — and the ASGI worker handling it — is blocked for the full timeout duration (often 60 s). Under concurrent load, all workers become occupied waiting on the same dead endpoint. Because there is no fast-fail path, the server stops serving *all* endpoints, not just LLM ones. The external API becomes a *single point of failure* for the entire application.

Part 2 — Architectural Remediation

Fix 1 · Optimistic Locking with Version Tokens

Add an integer **version** column to the document table. Every successful write increments it. Every read returns the current version to the client. On write, the client sends the version it last saw; the server issues a conditional UPDATE that only succeeds if the stored version still matches. If it does not match, the server returns **409 Conflict** and the client must re-fetch and re-apply its changes. This approach adds zero locking overhead in the happy path (hence 'optimistic') while guaranteeing that a stale write is always detected and rejected.

UML Sequence — Concurrent Edit Resolution

Alice	Server / DB	Bob
GET /doc/1 →	← {text:..., ver:5}	GET /doc/1 →
	← {text:..., ver:5}	
[edits locally]		[edits locally]

PUT ver=5 →	UPDATE WHERE ver=5 ✓	
	ver becomes 6	
		PUT ver=5 →
	UPDATE WHERE ver=5 ✗	
	409 Conflict →	[re-fetch, merge, retry]

Alice wins; Bob detects the conflict and retries with the merged state.

Fix 2 · Fault-Tolerant Webhook Handler

The webhook receiver must be made *durable and idempotent*. On receipt, immediately persist the raw event payload and its **Clerk event ID** to an **events** table with status *pending*. Return 200 OK right away — this acks the delivery and prevents Clerk from retrying unnecessarily. A background worker then processes pending events and marks them *processed*. Because the event ID is stored, replaying the same event is safe (no double-cancellation). A **dead-letter queue** captures events that fail processing after N retries, enabling manual inspection without data loss.

Fix 3 · Circuit Breaker with Fallback (Implemented)

Wrap every outbound LLM call in a **Circuit Breaker** with three states. **CLOSED** — requests flow normally. After *failure_threshold* consecutive failures, the circuit trips to **OPEN** — all subsequent calls return a pre-defined fallback message immediately, without touching the network, keeping server latency near zero. After *recovery_timeout* seconds, the circuit moves to **HALF_OPEN** and allows one probe request. Success resets to CLOSED; failure re-opens. This prevents cascading failures and keeps the server responsive to all other users even when the LLM provider is completely down.

State transitions:

CLOSED → (≥3 failures) → OPEN → (10 s timeout) → HALF_OPEN → (success) → CLOSED

CAP Theorem Trade-offs

Problem	Design Choice	CAP Trade-off
Lost Update (Optimistic Lock)	Reject stale writes; client retries	Favour Consistency over Availability: Conflicting writes are rejected (409) rather than silently merged.
Webhook (Idempotency + DLQ)	Persist-first, process async	Favour Consistency over Latency: Event is durably stored before acknowledging — never lost.
LLM Call (Circuit Breaker)	Fail-fast with cached fallback	Favour Availability over Consistency: Users get a stale fallback rather than an error or a 60-second wait.

No distributed system can achieve perfect Consistency, Availability, and Partition-tolerance simultaneously (Brewer's theorem). StudySync's fixes make deliberate, problem-specific trade-offs: correctness is non-negotiable for document edits; uptime is non-negotiable for the AI tutor. The webhook handler prioritises durability because a missed cancellation has direct revenue and compliance implications.